UADE

UADE

Programación III – Diseño y Análisis de Algoritmos

Temas a desarrollar

1. Introducción Programación Dinámica
2. Problema de Fibonacci
3. Problema del Cambio
4. Problema de la

Programación dinámica

Programación dinámica

- La técnica de **programación dinámica** se basa en resolver problemas en función de subproblemas más pequeños del mismo tipo.

- En **programación dinámica**, los subproblemas que se van construyendo deben ser dependientes debido la estrategia de resolución se basa en subproblemas comunes, para lo cual se resuelven los subproblemas y se almacena la solución para utilizarla nuevamente cuando sea necesario.

- Se utiliza en general para problemas de optimización.

Programación dinámica – Principio optimalidad

- La **programación dinámica** basa su funcionamiento en la aplicación del principio de optimalidad que dice que una solución puede ser óptima solo si está basada en soluciones parciales también óptimas.

- Si el principio de optimalidad no se puede aplicar para un determinado problema, no será posible entonces aplicar la técnica de **programación dinámica**.

Programación dinámica

En general se puede decir que los pasos a seguir para plantear una solución por Programación Dinámica son:

Plantear los problemas de tamaño inicial y soluciones directas.

Plantear la solución como una decisión entre soluciones menores.



Definición función de Fibonacci

Programación dinámica – Fibonacci

El problema de Fibonacci está dado por la definición misma de la función según se muestra a continuación:

$$F(n) = \begin{cases} 1 & \text{si } n < 2 \\ F(n-1) + F(n-2) & \text{sino} \end{cases}$$

Programación dinámica – Fibonacci

- Existe una solución a este problema por la **técnica de divide y conquista,** que no se considera aceptable en términos de complejidad temporal, ya que es exponencial debido a que un mismo valor se calcula varias veces, sin poder ser reutilizado.

- El **caso de recalcular un mismo valor varias veces** se debe a que los subproblemas no eran independientes como para ser tratados por divide y conquista. Por ello, aplicando la estrategia de programación dinámica para su resolución, se debería solucionar esa problemática y arribar a una solución mejor en cuanto a complejidad temporal.

Programación dinámica – Fibonacci - Estrategia

- En lugar de calcular el mismo valor varias veces, por ejemplo, para *F(5)* y *F(4)* se debe calcular *F(3)* en ambos casos, se va a almacenar el valor calculado para utilizarlo cuando se requiera. De esta manera se evitan operaciones duplicadas.

- Entonces podríamos almacenar los valores en una tabla como sigue:

$F(0)$	$F(1)$	$F(2)$	$\cdots$	$F(n)$

- Por lo tanto, en forma recurrente se puede definir como:

$$M[i] \begin{cases} 1 & si\ i < 2 \\ M[i-1] + M[i-2] & si\ i \geq 2 \end{cases}$$

Programación dinámica – Fibonacci - Algoritmo

```
ALGORITMO FIBBONACI
Entrada: n: entero
Salida: r: entero
  si n < 2
    devolver 1
  sino
    Vector<entero> tabla ← inicializarVector(n)
    tabla[0] ← 1
    tabla[1] ← 1
    para i ← 2 hasta n
      tabla[i] ← tabla[i − 1] + tabla[i − 2]
    fin para
    devolver tabla[n]
  fin si
```

Fibonacci – Complejidad Temporal

```
ALGORITMO FIBBONACI
Entrada: n: entero
Salida: r: entero
  si n < 2
    devolver 1
  sino
    Vector<entero> tabla ← inicializarVector(n)
    tabla[0] ← 1
    tabla[1] ← 1
    para i ← 2 hasta n
      tabla[i] ← tabla[i − 1] + tabla[i − 2]
    fin para
    devolver tabla[n]
  fin si
```

$$\mathcal{O}(n)$$

Problema del cambio

Problema del cambio - Definición

"Se tiene un conjunto de ***n*** denominaciones de monedas, de las cuales hay cantidad infinita por cada una de esas denominaciones. Dado un valor ***v*** se debe devolver la menor cantidad de monedad que cubran ese valor".

Nota: el problema debe ser válido para cualquier conjunto de denominaciones y cualquier importe.

Problema del cambio - Estrategia

- La estrategia por programación dinámica se puede plantear como:

$$C[i,j] \in \begin{cases} 0 & \text{si } j = 0 \\ +\infty & \text{si } i = 1 \text{ y } d_i > j \\ 1 + C[i, j - d_i] & \text{si } i = 1 \text{ y} d_i \leq j \\ c[i-1, j] & \text{si } i > 1 \text{ y } d_i > j \\ \text{mín}(1 + C[i, j - d_i], c[i-1, j]) & \text{sino} \end{cases}$$

- Para memorizar todos estos valores, se deberá construir una tabla con ***n*** filas, una para cada denominación y ***v +1*** columnas, una para cada valor posible entre ***0*** y ***v***. En cada posición (i; j) de la tabla estará la menor cantidad de monedas necesarias para devolver un valor ***j*** considerando solo ***i*** denominaciones, desde la ***1*** hasta la ***i***.

- La solución al problema será el valor que esta almacenado en la posición *(n; v)* de la tabla.

Problema del cambio - Ejemplo

- Para el caso de denominaciones {\$6, \$4, \$1} y debemos devolver por ejemplo \$8:

$$C[i,j] \in \begin{cases} 0 & \text{si } j = 0 \\ +\infty & \text{si } i = 1 \text{ y } d_i > j \\ 1 + C[i, j - d_i] & \text{si } i = 1 \text{ y} d_i \leq j \\ c[i-1, j] & \text{si } i > 1 \text{ y } d_i > j \\ \text{mín}(1 + C[i, j - d_i], c[i-1, j]) & \text{sino} \end{cases}$$

Valor(J)/ Monedas(I)	0	1	2	3	4	5	6	7	8
1	0 (j=0)	1 1+C[i,j-d1] por i=1 y di <=j	2 1+C[i,j-d1] por i=1 y di <=j	3 1+C[i,j-d1] por i=1 y di <=j	4 1+C[i,j-d1] por i=1 y di <=j	5 1+C[i,j-d1] por i=1 y di <=j	6 1+C[i,j-d1] por i=1 y di <=j	7 1+C[i,j-d1] por i=1 y di <=j	8 1+C[i,j-d1] por i=1 y di <=j
4	0 (j=0)	1 C[i-1,j] por i>1 y di >j	2 C[i-1,j] por i>1 y di >j	3 C[i-1,j] por i>1 y di >j	1 min (1+C[i, j-di]=1, C[i-1,j]=4)	2 min (1+C[i, j-di]=2, C[i-1,j]=5)	3 min (1+C[i, j-di]=3, C[i-1,j]=6)	4 min (1+C[i, j-di]=4, C[i-1,j]=7)	2 min (1+C[i, j-di]=2, C[i-1,j]=8)
6	0 (j=0)	1 C[i-1,j] por i>1 y di >j	2 C[i-1,j] por i>1 y di >j	3 C[i-1,j] por i>1 y di >j	1 C[i-1,j] por i>1 y di >j	2 C[i-1,j] por i>1 y di >j	1 min (1+C[i, j-di]=1, C[i-1,j]=3)	2 min (1+C[i, j-di]=2, C[i-1,j]=4)	2 min (1+C[i, j-di]=3, C[i-1,j]=2)

UADE

Problema del cambio - Algoritmo

```
ALGORITMO CAMBIO
Entrada: d: Vector<entero>, v: entero
Salida: c: entero
  M ← inicializarMatriz( longitud(d), v+1)
  para i ← 0 hasta longitud(d) − 1
    M[i, 0] ← 0
    para j ← 1 hasta v
      si i = 0
        si d[i] > j
          M[i, j] ← ∞
        sino
          M[i, j] ← M[i, j − d[j]] + 1
        fin si
      sino
        si d[j] > j
          M[i, j] ← M[i − 1, j]
        sino
          M[i, j] ← mín(M[i − 1, j], M[i, j − d[i] ] + 1))
        fin si
      fin si
    fin para
  fin para
  devolver M[longitud(i) − 1, v]
```

Problema del cambio – Complejidad temporal

```
ALGORITMO CAMBIO
Entrada: d: Vector<entero>, v: entero
Salida: c: entero
  M ← inicializarMatriz( Longitud(d), v+1)
  para i ← 0 hasta longitud(a) − 1
    M[i, 0] ← 0
    para j ← 1 hasta v
      si i = 0
        si d[i] > j
          M[i, j] ← ∞
        sino
          M[i, j] ← M[i, j − d[j]] + 1
        fin si
      sino
        si d[i] > j
          M[i, j] ← M[i − 1, j]
        sino
          M[i, j] ← mín(M[i − 1, j], M[i, j − d[i] ] + 1))
        fin si
      fin si
    fin para
  fin para
  devolver M[longitud(i) − 1, v]
```

Θ (n * v)

Problema del cambio – Greedy vs prog. dinámica

- La solución por Greedy, que era limitada en los valores a aceptar, tenía una complejidad temporal de (representando ***n*** el valor a devolver):

$$\mathcal{O}(n)$$

- La solución por programación dinámica, válida para toda instancia de valores, tiene una complejidad temporal de (representando ***n*** la cantidad de denominaciones y ***v*** valor a devolver):

$$\Theta\ (n * v)$$

- La solución por Greedy es más eficiente, aunque es limitada en cuanto a los casos que se pueden resolver “correctamente”, por lo cual solo una comparación de complejidad temporal no aplica en este caso.

Problema de la mochila

Problema de la mochila - Definición

- Se tienen ***n*** objetos y una mochila. Para ***i = 1; 2; ..;n***, el objeto ***i*** tiene un peso positivo ***pi*** y un valor positivo ***vi***.

- La mochila puede llevar un peso que no sobrepase P.

- Los objetos **no pueden ser fraccionados**.

- El objetivo es llenar la mochila de manera tal que se maximice el valor de los objetos transportados, respetando la limitación de capacidad impuesta.

Nota: la versión Greedy permitía fraccionar objetos.

Problema de la mochila - Estrategia

- La estrategia por programación dinámica se puede resumir como:

$$V[o,p] \in \begin{cases} 0 & \text{si } o = 1 \text{ y } peso[o] > p \\ valor[o] & \text{si } o = 1 \text{ y } peso[o] \leq p \\ V[o-1,p] & \text{si } o > 1 \text{ y } peso[o] > p \\ \text{máx}(V[o-1,p], V[o-1,p-peso[o]] + valor[o]) & \text{sino} \end{cases}$$

- En la matriz ***V*** tendremos una fila por cada objeto y una columna por cada peso posible entre cero y el máximo peso de la mochila.

Problema de la mochila - Ejemplo

- Supongamos un ejemplo con cuatro objetos con pesos (1, 4, 5 y 7) respectivamente y valores (10, 3, 5 y 8) respectivamente y una mochila con capacidad máxima de 8.

$$V[o,p] \in \begin{cases} 0 & \text{si } o = 1 \text{ y } peso[o] > p \\ valor[o] & \text{si } o = 1 \text{ y } peso[o] \leq p \\ V[o-1,p] & \text{si } o > 1 \text{ y } peso[o] > p \\ \text{máx}(V[o-1,p], V[o-1,p-peso[o]] + valor[o]) & \text{sino} \end{cases}$$

o	Vo	Po	0	1	2	3	4	5	6	7	8
1	10	1									
2	3	4									
3	5	5									
4	8	7									

Problema de la mochila - Ejemplo

$$V[o,p] \in \begin{cases} 0 & \text{si } o = 1 \text{ y } peso[o] > p \\ valor[o] & \text{si } o = 1 \text{ y } peso[o] \leq p \\ V[o-1,p] & \text{si } o > 1 \text{ y } peso[o] > p \\ \text{máx}(V[o-1,p], V[o-1,p-peso[o]] + valor[o]) & \text{sino} \end{cases}$$

o	Vo	Po	0	1	2	3	4	5	6	7	8
1	10	1	0 o=1 y P[o]>pi ->0	10 o=1 y P[o]<=pi -> Vo	10 o=1 y P[o]<=pi -> Vo	10 o=1 y P[o]<=pi -> Vo	10 o=1 y P[o]<=pi -> Vo	10 o=1 y P[o]<=pi -> Vo	10 o=1 y P[o]<=pi -> Vo	10 o=1 y P[o]<=pi -> Vo	10 o=1 y P[o]<=pi -> Vo
2	3	4	0 o>1 y P[o]>pi->V[o-1,p]	10 o>1 y P[o]>pi->V[o-1,p]	10 o>1 y P[o]>pi->V[o-1,p]	10 o>1 y P[o]>pi->V[o-1,p]	10 o>1 y P[o]<=pi->max(V[o-1,p]=10,V[o-1,p-P[o]]+V[o]=3)	13 o>1 y P[o]<=pi->max(V[o-1,p]=10,V[o-1,p-P[o]]+V[o]=13)	13 o>1 y P[o]<=pi->max(V[o-1,p]=10,V[o-1,p-P[o]]+V[o]=13)	13 o>1 y P[o]<=pi->max(V[o-1,p]=10,V[o-1,p-P[o]]+V[o]=13)	13 o>1 y P[o]<=pi->max(V[o-1,p]=10,V[o-1,p-P[o]]+V[o]=13)
3	5	5	0 o>1 y P[o]>pi->V[o-1,p]	10 o>1 y P[o]>pi->V[o-1,p]	10 o>1 y P[o]>pi->V[o-1,p]	10 o>1 y P[o]>pi->V[o-1,p]	10 o>1 y P[o]>pi->V[o-1,p]	13 o>1 y P[o]<=pi->max(V[o-1,p]=13,V[o-1,p-P[o]]+V[o]=5)	15 o>1 y P[o]<=pi->max(V[o-1,p]=13,V[o-1,p-P[o]]+V[o]=15)	15 o>1 y P[o]<=pi->max(V[o-1,p]=13,V[o-1,p-P[o]]+V[o]=15)	15 o>1 y P[o]<=pi->max(V[o-1,p]=13,V[o-1,p-P[o]]+V[o]=15)
4	8	7	0 o>1 y P[o]>pi->V[o-1,p]	10 o>1 y P[o]>pi->V[o-1,p]	10 o>1 y P[o]>pi->V[o-1,p]	10 o>1 y P[o]>pi->V[o-1,p]	10 o>1 y P[o]>pi->V[o-1,p]	13 o>1 y P[o]>pi->V[o-1,p]	13 o>1 y P[o]>pi->V[o-1,p]	15 o>1 y P[o]<=pi->max(V[o-1,p]=15,V[o-1,p-P[o]]+V[o]=8)	18 o>1 y P[o]<=pi->max(V[o-1,p]=15,V[o-1,p-P[o]]+V[o]=18)

UADE

Problema de la mochila - Algoritmo

ALGORITMO MOCHILA
Entrada: O: Vector<Objeto>, P: entero
Salida: v: entero
  $M \leftarrow inicializarMatrix(longitud(O), P+1)$
  para $i = 0$ **hasta** $longitud(O) - 1$
    para $j = 0$ **hasta** P
      si $i = 0$ **Y** $peso[i] > j$
        $M[i, j] \leftarrow 0$
      sino
        si $i = 0$
          $M[i, j] \leftarrow valor[i]$
        sino
          si $peso[i] > j$
            $M[i, j] \leftarrow M[i-1, j]$
          sino
            $M[i, j] \leftarrow \text{máx}(M[i-1, j], M[i-1, j - peso[i]] + valor[i])$
          fin si
        fin si
      fin si
    fin para
  fin para
  devolver $M[longitud(O) - 1, P]$

Problema de la mochila – Complejidad temporal

```
ALGORITMO MOCHILA
Entrada: O: Vector<Objeto>, P: entero
Salida: v: entero
  M ← inicializarMatrix(longitud(O), P + 1)
  para i = 0 hasta longitud(O) − 1
    para j = 0 hasta P
      si i = 0 Y peso[i] > j
        M[i, j] ← 0
      sino
        si i = 0
          M[i, j] ← valor[i]
        sino
          si peso[i] > j
            M[i, j] ← M[i − 1, j]
          sino
            M[i, j] ← máx(M[i − 1, j], M[i − 1, j − peso[i]] + valor[i])
          fin si
        fin si
      fin si
    fin para
  fin para
  devolver M[longitud(O) − 1, P]
```

Θ (O * P)

¡Muchas gracias!

UADE